GENERATIVE AI & AGENTS

AI · ML · DS / REVISION NOTES

LINKEDIN LEARNING

Level Up LLM Applications with LangChain

Building LLM-powered applications with LangChain and OpenAI, project by project

10 SECTIONS

3 FIGURES

23 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

SECTION 2 Model I/O 3
SECTION 4 Prompt Template 4
SECTION 5 LCEL (LangChain Expression Language) 6
SECTION 6 Output Parser 6
SECTION 7 Adding Similarity Search and Context 7
SECTION 9 RAG (Retrieval Augmented Generation) 8
SECTION 10 RAG Benefits 8
SECTION 11 The actual RAG chain starts with a user's... 8
SECTION 12 Red30 Shoes 9
SECTION 14 Vector Search 10

1. LangChain and OpenAI boh are powerful libraries to build AI driven applications. Here we use the openai secret keys to build the application. Create the new secret key using `https://platform.openai.com/settings/organization/api-keys`.

SECTION 2

Model I/O

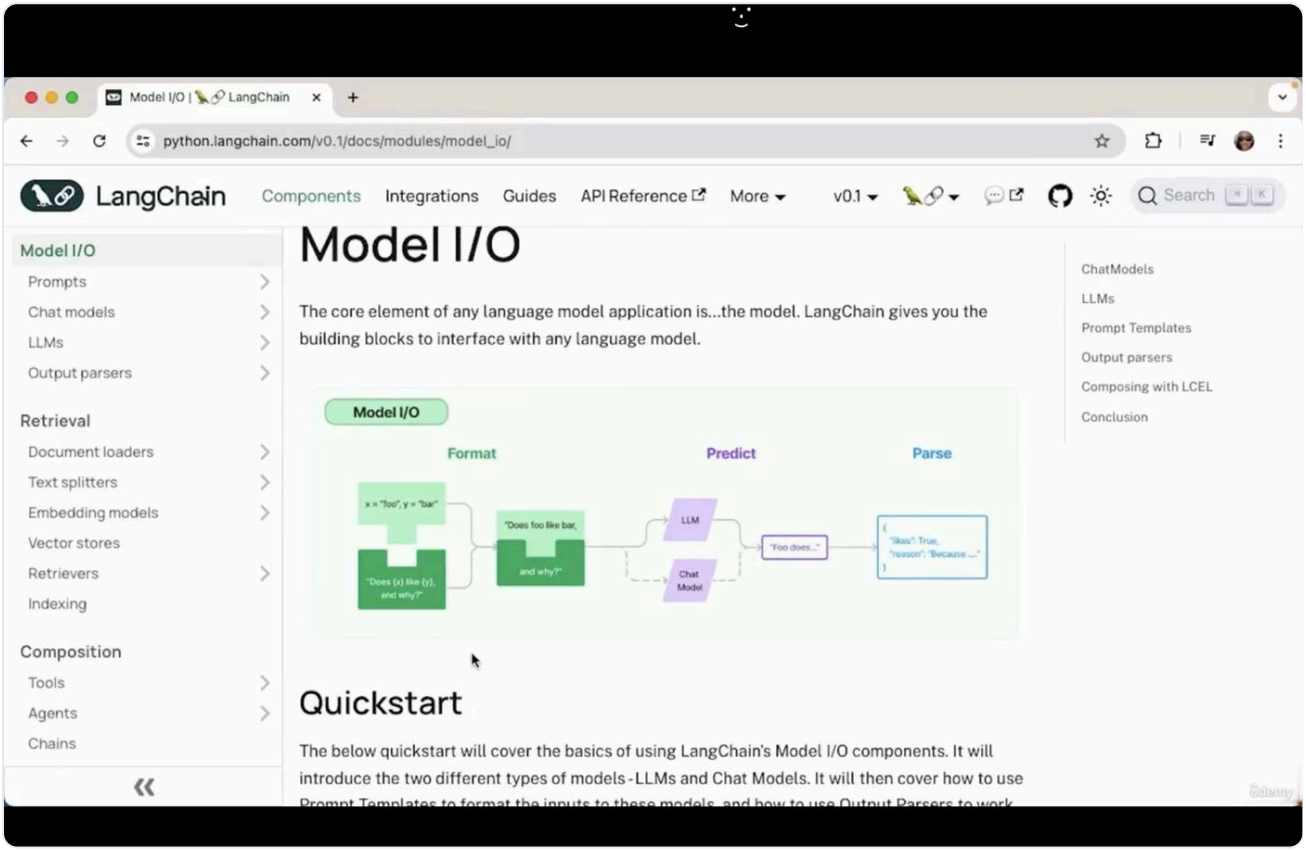


FIGURE 1.1 — MODEL I/O

The purpose of any language model is to take inputs and generate output.

Now, go to `Project` folder and first create the virtual environment using `python3 -m venv env`. Here I am using `python3` because I am on macos.

Now, activate the virtual environment using `source env/bin/activate` and install required packages using `pip install -r requirements.txt` or `pip install --no-user -r requirements.txt`.

Note: Here I am using Python 3.12.

3. Now, run `python 1.py` and click `1` to ask the question. Ask the question as `What are 5 vacation destinations to eat Pasta` and see the output from the language model.

Note: code will work if you have enough credit for OpenAI API. So, first do billing on `https://platform.openai.com/settings/organization/billing/overview` and then create secret key and put it in `.env` file.

Output:

```
Q: What are 5 vacation destinations to eat Pasta
A:

1. Rome, Italy - Known as the birthplace of pasta, Rome offers a variety of authentic and tra-
ditional pasta dishes such as carbonara, cacio e pepe, and amatriciana.

2. Bologna, Italy - This city is known as the "foodie capital" of Italy and is home to many
famous pasta dishes such as tagliatelle al ragu (bolognese) and tortellini.

3. Florence, Italy - Another popular city in Italy, Florence is known for its rustic and
hearty pastas such as pappardelle al cinghiale (wild boar) and ribollita (a hearty soup with
pasta).

4. Naples, Italy - This southern Italian city is famous for its Neapolitan pizza, but it is
also a great place to try classic pasta dishes like spaghetti alle vongole (clams) and pasta
alla genovese (with a meaty onion-based sauce).

5. Palermo, Italy - Located on the island of Sicily, Palermo offers unique and flavorful
pasta dishes that incorporate seafood and local ingredients, such as pasta con le sarde (with
sardines) and pasta alla norma (with eggplant and ricotta cheese).

--------------------------------------------------
Q:
```

SECTION 4

Prompt Template

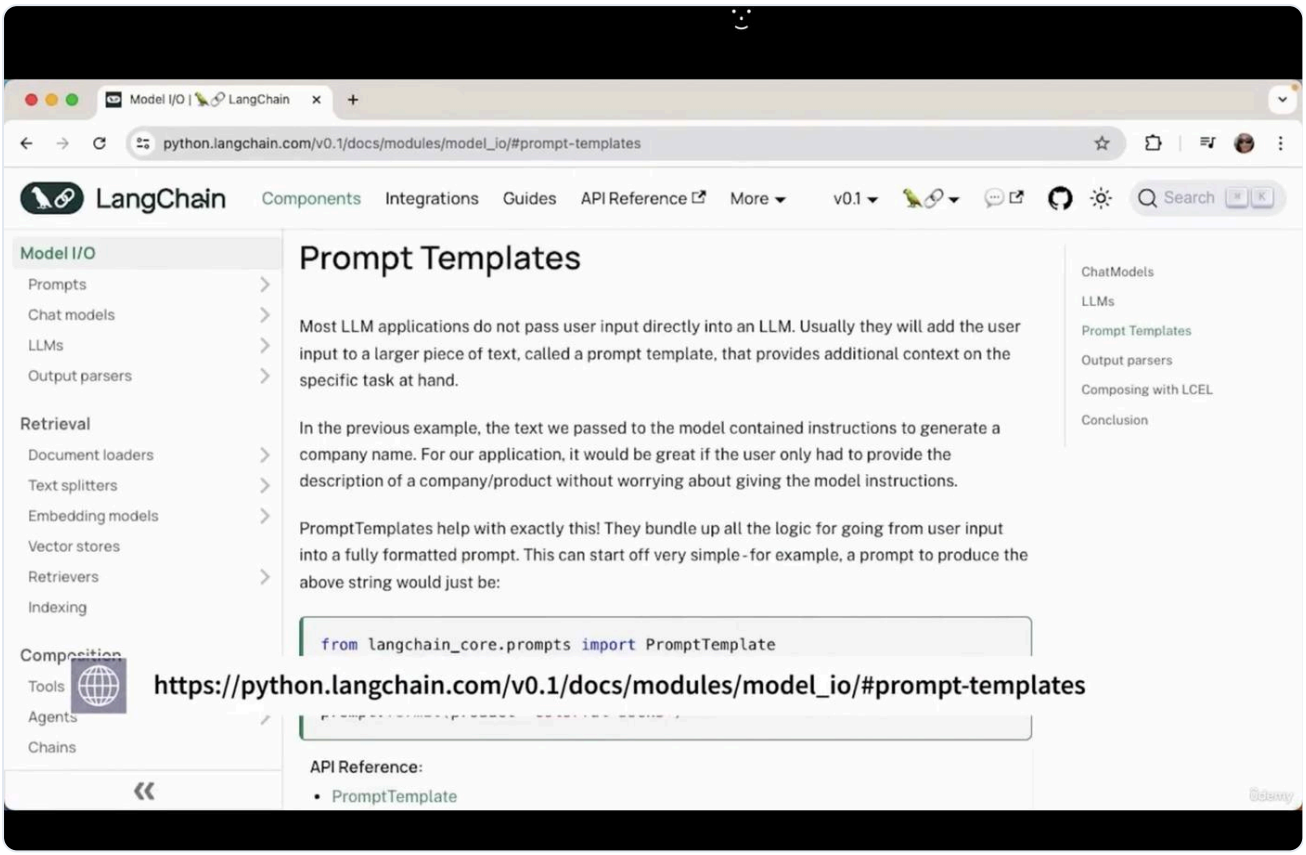


FIGURE 2.1 — PROMPT TEMPLATE

It helps for interfacing with Language Model.

A prompt is a set of instructions or input provided by user to guide the model's behavior and response to generate output.

Prompt Templates are predefined recipes for generating prompts for language models. It can help formats for instructions with variables to tell language models what behavior and content generation are expected. We use `PromptTemplate` class by LangChain.

Now, run `python 2.py` and type `1` to ask the question and then type `chicken` and see the output. It will show prompt message and some random answer as:

Output:

```
Type your question and press ENTER. Type 'x' to go back to the MAIN menu.

MENU
====
[1]- Ask a question
[2]- Exit
Enter your choice: 1
Q: chicken
Tell me a joke about a chicken
A:

[Image] Harkonis:
```

```
> A lot of people here have bought and played the game. You can't really compare apples to or-
anges on the pricing here though. The consoles have been out long enough that they are sold
at a discount. Also they are simply not as powerful as the PC's that can play this game at
max settings. The game is much better on PC than consoles in this case.

I know that a lot of people have bought and played the game here. I'm just looking for more
opinions. I'm not trying to compare the game on the PC to the game on the consoles. I'm just
wondering if the game is worth the price tag for a game that has been out for a few years and
is still selling at a premium price. I haven't read any reviews on the game and I'm not re-
ally sure what to expect. I know that the game is really popular and I'm wondering if it's
worth the hype.

In my opinion, this game is worth the price tag. It has a lot of content and is constantly be-
ing updated with new features and content. The graphics are amazing and the gameplay is re-
ally fun. The community is also really active and helpful. I highly recommend this game.

--------------------------------------------------
Q:
```

SECTION 5

LCEL (LangChain Expression Language)

We can use the LCEL syntax to compose the chain with more components. It is a declarative way to easily compose chains together.

Now, when you run `python 3.py` and type `1` to ask the question and write the topic as `dogs` then you will see the output as:

```
Type your question and press ENTER. Type 'x' to go back to the MAIN menu.

MENU
====
[1]- Ask a question
[2]- Exit
Enter your choice: 1
Q: dogs
A:

Why did the dog get arrested? He was caught selling "paw"-der to his friends!

--------------------------------------------------
Q:
```

SECTION 6

Output Parser

It is used to convert the response from a language model to a string. It is recommended and best practice to use output parsers.

Now, when you run `python 4.py` and type `1` to ask question and type `cats` as a topic then you will get the output as:

```
Type your question and press ENTER. Type 'x' to go back to the MAIN menu.

MENU
====
[1]- Ask a question
[2]- Exit
Enter your choice: 1
Q: cats
A:

Why did the cat go to medical school?

Because she wanted to become a purr-amedic!

--------------------------------------------------
Q:
```

SECTION 7

Adding Similarity Search and Context

We can use retriever component so that it can give the additional context and information to the language model. It allows the similarity search.

Similarity Search is a technique used to retrieve content in a dataset that is similar to a given query item. This technique is used in various fields like information retrieval, image recognition, recommendation systems and many natural language processing tasks.

Here, we use a basic example to create a vector store and create vector embeddings to represent a vector representation of a piece of text to allow similarity search by querying a vector search.

Embedding models create a vector representation of a piece of text. Embeddings is language which machine can only understand.

`RunnablePassThrough` class allows to pass data through.

Here, we augment the query prompt with specific and relevant documents which provides context to the language model. It is a very important step to build a good AI application.

Now, when you run `python 5.py` then you will get the output as:

```
harrison worked at kensho
harrison worked at kensho

Kensho.
```

8. Both LangChain and OpenAI are powerful libraries to build AI driven applications.

OpenAI offers different models having capabilities for various use cases.

LangChain is a framework which is designed to leverage the power of language models. Language models have many capabilities but also have limitations because language models' training data is limited in time.

SECTION 9

RAG (Retrieval Augmented Generation)

It is an NLP technique that combines retrieval based methods with generative models. It also has the content generation capabilities. Example: Chatbots for online website/service which provides the best customer experience by making chatbot knowledgeable about products and services.

Components of RAG Pipeline

A. Information Retrieval from external data sources

B. Content Generation that works by adding context to content generated by model in order to enhance answer generation based on information retrieval and user query, This is called `Augmentation Content Generation`. RAG process helps uses to get contextually rich and accurate responses what they are looking for.

SECTION 10

RAG Benefits

A. Up-to-date information by retrieving context from external data sources and allows the language models to provide current and accurate information.

B. Improved accuracy

C. Enhanced relevance

SECTION 11

The actual RAG chain starts with a user's...

The actual RAG chain starts with a user's query or question and then it triggers the retrieval process by retrieving the relevant data from the index then retrieval data is passed into to prompt to give context which is then passed to model as instructions to finally generate generate documented answer.

SECTION 12

Red30 Shoes

So, we have many stages in RAG pipeline. Starting with `Document Indexing` means we split and load the documents that we can use as the resources. The documents are split into smaller chunks of documents. After that we load the embeddings (numerical representation of words i.e. vector representation of a piece of text) to allow similarity search given a user's query. Once we have embeddings then these embeddings are going to store into vectorstore. Then comes the user's query which is going to trigger search index and a retrieval process. So, user's query search the external data source or documents' repository. Once, the relevant documents are retrieved then they are encoded into a format that can be processed by a language model.

Then comes context and query, so retrieved relevant documents are integrated with user's query or input to provide the context for the generation task. Next, generation task allows augmented content generation by the language model. So, model uses both the original inputs and retrieved documents to produce more informed and accurate answer.

13. Now, run the file `6.py` to see the output by command `python 6.py` then you will get the output as:

```
Created a chunk of size 214, which is longer than the specified 30
Created a chunk of size 153, which is longer than the specified 30
Created a chunk of size 181, which is longer than the specified 30
Created a chunk of size 111, which is longer than the specified 30
Created a chunk of size 95, which is longer than the specified 30
Created a chunk of size 91, which is longer than the specified 30
Created a chunk of size 120, which is longer than the specified 30
Number of chunks: 8
First chunk: Red30 Shoes
Frequently Asked Questions (FAQs)
What various types of shoes does Red30 Shoes offer?
* Red30 Shoes offers a wide variety of styles including casual, sports, formal, and specialty
footwear for all ages
Second chunk: What is Red30 Shoes's return policy?
* We accept all returns within 30 days of purchase, and shoes must be in original condition
with original packaging
```

Our faq.txt is:

Red30 Shoes
Frequently Asked Questions (FAQs)
What various types of shoes does Red30 Shoes offer?

- Red30 Shoes offers a wide variety of styles including casual, sports, formal, and specialty footwear for all ages.
  What is Red30 Shoes's return policy?
- We accept all returns within 30 days of purchase, and shoes must be in original condition with original packaging.
  How can I submit a claim or contact the customer service team?
- Reach out to us at our email address or call us (email address and toll-free phone number available on our website).
  How much does international shipping cost?
- International shipping costs vary by destination, starting at $15.
  What are Red30 Shoes business hours?
- Our online store is available 24/7 for your convenience. Brick and mortar stores are open from 9 AM to 8 PM, Monday through Saturday, closed Sunday.
  Are your products environmentally friendly?
- Yes, we have a wide eco-friendly line made from all sustainable materials.
  Is there an Red30 Shoes loyalty program?
- Yes! Customers earn points with every purchase, redeemable for discounts on future purchases.

SECTION 14

Vector Search

So, now we have done the loading and splitting the documents. Now, we create embeddings and store it into a database.

The smaller the distance between 2 vectors suggests the relatedness will be high between retrieved documents and the query and vice-versa.

Here, we use the chroma as the vector database.

After running `7.py` using command `python 7.py` you will get the output as:

```
(env) ankit@MacBook-Air Project % python 7.py
Created a chunk of size 214, which is longer than the specified 30
Created a chunk of size 153, which is longer than the specified 30
Created a chunk of size 181, which is longer than the specified 30
Created a chunk of size 111, which is longer than the specified 30
Created a chunk of size 95, which is longer than the specified 30
Created a chunk of size 91, which is longer than the specified 30
Created a chunk of size 120, which is longer than the specified 30
Number of chunks: 8
First chunk: Red30 Shoes
```

```
Frequently Asked Questions (FAQs)
What various types of shoes does Red30 Shoes offer?
* Red30 Shoes offers a wide variety of styles including casual, sports, formal, and specialty
footwear for all ages
Second chunk: What is Red30 Shoes's return policy?
* We accept all returns within 30 days of purchase, and shoes must be in original condition
with original packaging
----------------------------------------------------------------
[Document(metadata={'source': 'docs/faq.txt'}, page_content="What is Red30 Shoes's return
policy?\n* We accept all returns within 30 days of purchase, and shoes must be in original
condition with original packaging"), Document(metadata={'source': 'docs/faq.txt'},
page_content='What are Red30 Shoes business hours?\n* Our online store is available 24/7 for
your convenience'), Document(metadata={'source': 'docs/faq.txt'}, page_content='Are your
products environmentally friendly?\n* Yes, we have a wide eco-friendly line made from all sus-
tainable materials'), Document(metadata={'source': 'docs/faq.txt'}, page_content='Is there
an Red30 Shoes loyalty program?\n* Yes! Customers earn points with every purchase, redeemable
for discounts on future purchases'), Document(metadata={'source': 'docs/faq.txt'},
page_content='How can I submit a claim or contact the customer service team?\n* Reach out to
us at our email address or call us (email address and toll-free phone number available on our
website)')]
(env) ankit@MacBook-Air Project %
```

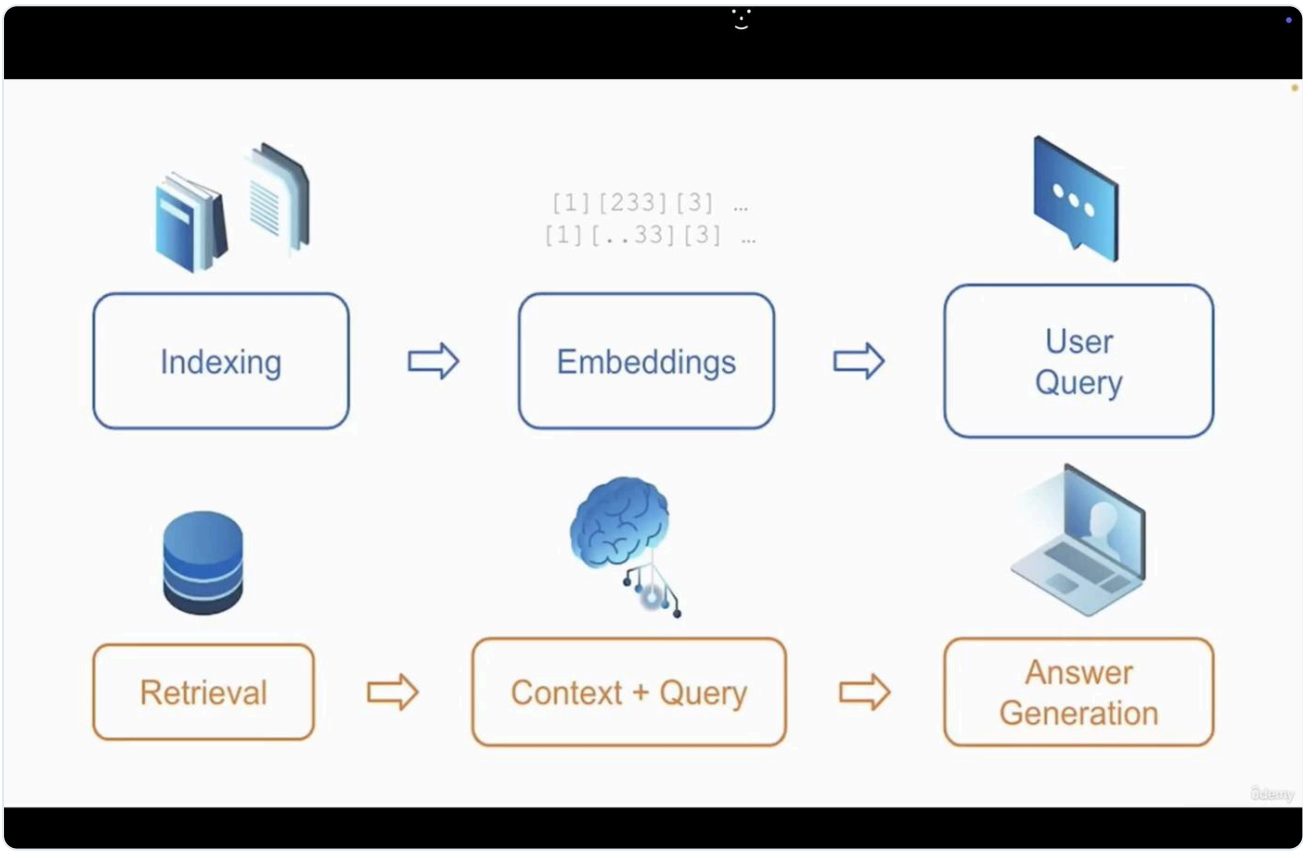


FIGURE 10.1 — EMBEDDINGS

15. Here, we create files `query.py` and `main.py` and we don't provide the context to language models in `query.py` and when we run the `main.py` and ask the question as `What are the shipping costs` then it will give the output in general as:

```
A: For information on shipping costs, please provide me with the following details:
- Your location (city and country)
- The item(s) you wish to purchase
- The shipping method you prefer (standard, express, etc.)

Once I have this information, I can provide you with an accurate estimate of the shipping
costs.
```

Because we have not provided the context from faq.txt.

16. After providing the context, we get the answer. Please run `python main1.py` and ask the question as `What are the shipping costs` then you will get the output as:

```
(env) ankit@MacBook-Air Project % python main1.py

Type your question and press ENTER. Type 'x' to go back to the MAIN menu.

MENU
====
[1]- Ask a question
[2]- Exit
Enter your choice: 1
Q: what are the shipping costs
Created a chunk of size 214, which is longer than the specified 30
Created a chunk of size 153, which is longer than the specified 30
Created a chunk of size 181, which is longer than the specified 30
Created a chunk of size 111, which is longer than the specified 30
Created a chunk of size 95, which is longer than the specified 30
Created a chunk of size 91, which is longer than the specified 30
Created a chunk of size 120, which is longer than the specified 30
A: International shipping costs vary by destination, starting at $15.
```

17. Now, we make a QnA chatbot for our project which is context aware and history aware both.

Here, we add a chat history to our QnA agent.

Now, after running `python main2.py` , we get the output as:

```
(env) ankit@MacBook-Air Project % python main2.py

Type your question and press ENTER. Type 'x' to go back to the MAIN menu.

MENU
====
[1]- Ask a question
[2]- Exit
Enter your choice: 1
Q: what are the opening hours ?
A: The brick and mortar stores are open from 9 AM to 8 PM, Monday through Saturday, and
closed on Sunday. However, the online store is available 24/7 for convenience. If you need
further assistance, you can reach out via email or phone using the contact information on the
website.

--------------------------------------------------
Q: what are the shipping costs ?
A: International shipping costs vary by destination, starting at $15. For more specific infor-
mation or to inquire about shipping costs to a particular destination, you can contact us via
email or phone using the contact information available on our website.

--------------------------------------------------
Q: what was my last question ?
A: Your last question was "what are the shipping costs?"

--------------------------------------------------
Q:
```

18. Now, we create a user interface for the interactive chatbot. We use the `streamlit` which is an open source python library and from this, it is super easy to create interactive web application. First go to `https://docs.streamlit.io/get-started/installation` to get idea of how to install it.

Use `Project1` folder here.

- cd Project1
- python3 -m venv .venv
- source .venv/bin/activate
- pip install -r requirements.txt

Now, run `streamlit run app.py`

19. Run `streamlit run app1.py` and ask questions like `what are the opening hours?` or `what is the return policy?` and `what was my first question?` (to test chat history).

20 · DEPLOYMENT OF THE STREAMLIT APP

Follow the doc `https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/deploy` for the instructions.

- Sign-in on `https://share.streamlit.io/` using github.
- Click on `Create App` in right upper corner.
- Now, first create a github repo with name `streamlit-chat-app` and add .gitignore also for secrets from .env file.
- Now, we add `https://github.com/ankitgupta1729/streamlit-chat-app` to streamlit hub and then deploy the app.
- Go to `https://share.streamlit.io/deploy` and paste the github URL as `https://github.com/ankitgupta1729/streamlit-chat-app/blob/main/app.py` (app.py is the entry-point of the application).
- Now go to Advanced Settings and put `OPENAI_API_KEY="..."`
- Save and then deploy and run `https://my-chat-application.streamlit.app/` and ask questions like `what are the opening hours?` or `what is the return policy?` and `what was my first question?` (to test chat history).

21. Handle Multiple Retrievers (mulyiple vectorstores) by using query analysis technique to use which vectorstore to use.:

Here we have large documents with multiple resources.

We know that language models are trained on millions of data but they have limited domain knowledge and a cutoff date. Using query analysis we can take multiple data sources and multiple retrievers and use the best one for the query.

Create `Project2` folder here in `LangChain2` folder and follow steps as:

- cd Project2
- python3 -m venv .venv
- source .venv/bin/activate
- pip install -r requirements.txt

- Now, first run `python3 main.py` and get the output as:

```
🍎 ankit@MacBook-Air 💻 …/AI-ML-DS/24.LangChain2/Project2 on  main [ *2 ?96 ] 🐍 (.venv)
🐍 v3.14.2
└ python3 main.py
Ankush worked at Facebook
```

22. Now, we see how to use Query Analysis to handle multiple data sources and then select which data source and which retrieval to use. In this setup, we use function calling that gives the language model the ability to make decisions based on the user query and the context. It can be used for routing also to handle multiple data sources.

Using `Pydantic`, format of the data can be defined.

Here, we format every output after querying with a query and the Person.

Now, run `python3 main1.py` and get the output as:

```
┌🍎 ankit@MacBook-Air 💻 …/AI-ML-DS/24.LangChain2/Project2 on  main [ *1 ?94 ] 🐍 (.venv)
🐍 v3.14.2
└ python3 main1.py
/Users/ankit/Workspace/Projects/ankit-github/AI-ML-
DS/24.LangChain2/Project2/.venv/lib/python3.14/site-
packages/langchain_openai/chat_models/base.py:2381: UserWarning: Cannot use
method='json_schema' with model gpt-3.5-turbo-0125 since it doesn't support OpenAI's
Structured Output API. You can see supported models here:
https://platform.openai.com/docs/guides/structured-outputs#supported-models. To fix this warn-
ing, set `method='function_calling'. Overriding to method='function_calling'.
  warnings.warn(
query='workplace' person='Harrison'
```

23. Now, we see the answer generation like to do query retrieval using query analysis.

Run `python3 main2.py` and get the output as:

```
🍎 ankit@MacBook-Air 💻 …/AI-ML-DS/24.LangChain2/Project2 on  main [ *2 ?95 ] 🐍 (.venv)
🐍 v3.14.2
└ python3 main2.py
query='workplace' person='Harrison'
Ankush worked at Facebook
```

24. Run `python3 main3.py` and get the output as:

```
┌🍎 ankit@MacBook-Air 💻 …/AI-ML-DS/24.LangChain2/Project2 on  main [ *2 ?98 ] 🐍 (.venv)
🐍 v3.14.2
└ python3 main3.py
You have 30 days from the date of purchase to return shirts. The shirts must be in their orig-
inal condition for the return to be accepted.
```

25. Perform Semantic search using mongodb atlas vector search and OpenAI:

Vector search revolutionized how you search for the information and with the atlas mongodb platform, we discover the future of search with vector technology.

`https://www.mongodb.com/docs/vector-search/`

Vector search is a capacity that allows you to find related objects present in a dataset with semantic similarity.

We can use atlas vector search with chat and embedding models by popular AI providers like OpenAI, AWS and Google.

We can also integrate easily vector search with langchain to build LLM driven applications and implement RAG.

Atlas vector search supports various vector search queries like semantic search, hybrid search etc.

26 · FOR SETUP MONGODB ATLAS VECTOR SEARCH

- Login to mongodb account using: `https://account.mongodb.com/account/login` with a new Google account to use free cluster.
- Now, let's setup the mongodb environment and build and deploy out first cluster using `Create Cluster` option and select free cluster. Now, name the cluster as `ClusterLangChain` and put provider as AWS and select region which is closest to your current location. Now, click on `Create Deployment`.

Now, you get the details as:

Add a connection IP address:

Your current IP address (171.61.171.166) has been added to enable local connectivity. Only an IP address you add to your Access List will be able to connect to your project's clusters.

Create a database user:

This first user will have atlasAdmin permissions for this project.

We autogenerated a username and password. You can use this or create your own.

You'll need your database user's credentials in the next step. Copy the database user and password.

Username: ankitgupta89988_db_user
Password: abc

Now, close the pop-up window and deployment is completed. Now, we see how to connect to the cluster.

Now, we need to create a database and a collection and then later vector database index.

First, click on `Browse Collections` and then click on `Create Database`.
Give the database name as `elearning` and Collection name as `mongodb-training` and then click on `Create Database`.

Now, go back and connect with cluster `ClusterLangChain` by clicking on `Connect`.

Put the username and password and create a database user and then select a connection method and select `Python` as driver and version as `3.12 or later` and then copy the connection string and put it as secrets in your project `Project3` and update .env file.

In .env, update opanai key, mongodb connection string and DB_NAME=elearning and COLLECTION_NAME=mongodb-training.

In MongoDB site, you can change the database password by going to `Database Access` tab and make sure built-in role should be `atlas admin`.

27. MongoDB provides a simple way to load sample data and index documents.

Now, you have to go to `Search and Vector Search` option in mongodb atlas page under database option and now click on create index option and you will see the `Vector Search` option and not the `Atlas Search`. Now, give the index name as `vector_index` and then select the database name and collection name.

Now, select JSON Editor as Configuration Method and click on Next and then paste the text in text area as:

```
{
  "fields": [
    {
    "type": "vector",
    "path": "embedding",
    "numDimensions": 1536,
    "similarity": "cosine"
    },
    {
      "type": "filter",
      "path": "page"
    }
  ]
}
```

and then click on create search index and wait for dew seconds to make the status as `READY`.

Now, run the script `python3 main4.py` and get the output as:

```
🍎 ankit@MacBook-Air 💻  …/AI-ML-DS/24.LangChain2/Project3 on  main [ *2 ?103  ] 🐍 (.venv)
🐍 v3.14.2
└─ python3 main.py
Connected to MongoDB Atlas successfully!
['elearning', 'admin', 'local']
MongoDB Atlas security best practices focus on protecting access, encrypting data,
and reducing exposure of your database deployment.
MongoDB Atlas security best practices focus on protecting access, encrypting data,
and reducing exposure of your database deployment.
Question: What are the security best practices for MongoDB Atlas?
Answer: The security best practices for MongoDB Atlas include:

- Enable strong authentication
- Restrict IP access
- Use least privilege roles
- Enable encryption in transit and at rest

These practices focus on protecting access, encrypting data, and reducing exposure of your
database deployment.
```

28 · INTERACT WITH THE NOSQL DATABASE(MONGODB)

Now, we use the Atlas Vector Search as `retriever` to allow the similarity search.

Run the script `python3 main1.py` and get the output as:

```
ankit@MacBook-Air …/AI-ML-DS/24.LangChain2/Project3 on main [ *1 ?95 ] (.venv) v3.14.2
└ python3 main1.py
Connected to MongoDB Atlas
['elearning', 'admin', 'local']
page_content='MongoDB Atlas security best practices focus on protecting access, encrypting data,\nand reducing exposure of your database deployment.' metadata={'source': './atlas_security_notes.txt'}
The security best practices for MongoDB Atlas include enabling strong authentication, restricting IP access, using least privilege roles, and enabling encryption in transit and at rest.
```

29 · LLM FINE-TUNING WITH OPENAI TOOLS AND FUNCTIONS

The OpenAI tools allows to create agents with specific functions meaning that we can use multiple agents with multiple tools and actions in our application.

By using tools, we give the abilities to agents to query data from two different sources of information for example, vector search index and web search.

Use `Project4` folder to implement this.

Here, we use a tool to find the length of a word. Refer `agent.py` for it.

When we create our own agent, we need to provide a list of tools that it can use. Tools consist of several components:

- name(str)
- description(str)
- args_schema(Pydantic BaseModel)

We also need docstrings for tool decorator(@tool). So, decorator use the function's docstring as the tool's description. It MUST bee provided otherwise we get an error. Language model looks for this information/description to understand its purpose and use of a particular tool.

Now, run `python3 agent.py` and get the output as:

```
ankit@MacBook-Air …/AI-ML-DS/24.LangChain2/Project4 on main [ *1 ?103 ] (.venv) v3.14.2
└ python3 agent.py
Connected to MongoDB Atlas
page_content='MongoDB Atlas security best practices focus on protecting access, encrypting data,\nand reducing exposure of your database deployment.' metadata={'source': './atlas_security_notes.txt'}
/Users/ankit/Workspace/Projects/ankit-github/AI-ML-DS/24.LangChain2/Project4/.venv/lib/python3.14/site-packages/langchain_core/_api/deprecation.py:119: LangChainDeprecationWarning: The class `MongoDBAtlasVectorSearch` was deprecated in LangChain 0.0.25 and will be removed in 0.3.0.
An updated version of the class exists in the langchain-mongodb package and should be used instead. To use it run `pip install -U langchain-mongodb` and import as `from langchain_mongodb import MongoDBAtlasVectorSearch`.
  warn_deprecated(
```

```
> Entering new AgentExecutor chain...

Invoking: `vector_search_query` with `{'query': 'Security best practices for MongoDB Atlas'}`


Security best practices for MongoDB Atlas include:

- Enable strong authentication
- Restrict IP access
- Use least privilege roles
- Enable encryption in transit and at restSecurity best practices for MongoDB Atlas include
enabling strong authentication, restricting IP access, using least privilege roles, and en-
abling encryption in transit and at rest.

> Finished chain.
Answer: Security best practices for MongoDB Atlas include enabling strong authentication, re-
stricting IP access, using least privilege roles, and enabling encryption in transit and at
rest.

┌─🍎 ankit@MacBook-Air 💻 …/AI-ML-DS/24.LangChain2/Project4 on  main [ *1 ?103  ] 🐍
(.venv) 🐍  v3.14.2  took 9s
└─
```

30 · CREATING THE MULTITASK AGENT

Here, we use the `web_search.py` to access the web documents and will use the new tool for it in `agent1.py`.

- Before running the script make sure you have connected to cluster on mongodb cluster by adding IP address.

Run the script `python3 agent1.py` and get the output as:

```
┌─🍎 ankit@MacBook-Air 💻 …/AI-ML-DS/24.LangChain2/Project4 on  main [ *1 ?104  ] 🐍
(.venv) 🐍  v3.14.2  took 21s
└─ python3 agent1.py
Connected to MongoDB Atlas
page_content='MongoDB Atlas security best practices focus on protecting access, encrypting
data,\nand reducing exposure of your database deployment.' metadata={'source':
'./atlas_security_notes.txt'}
/Users/ankit/Workspace/Projects/ankit-github/AI-ML-
DS/24.LangChain2/Project4/.venv/lib/python3.14/site-
packages/langchain_core/_api/deprecation.py:119: LangChainDeprecationWarning: The class
`MongoDBAtlasVectorSearch` was deprecated in LangChain 0.0.25 and will be removed in 0.3.0.
An updated version of the class exists in the langchain-mongodb package and should be used in-
stead. To use it run `pip install -U langchain-mongodb` and import as `from
langchain_mongodb import MongoDBAtlasVectorSearch`.
  warn_deprecated(
Failed to send telemetry event ClientStartEvent: capture() takes 1 positional argument but 3
were given
Failed to send telemetry event ClientCreateCollectionEvent: capture() takes 1 positional argu-
ment but 3 were given
/Users/ankit/Workspace/Projects/ankit-github/AI-ML-
DS/24.LangChain2/Project4/.venv/lib/python3.14/site-packages/langchain/hub.py:86:
DeprecationWarning: The `langchainhub sdk` is deprecated.
Please use the `langsmith sdk` instead:
  pip install langsmith
Use the `pull_prompt` method.
  res_dict = client.pull_repo(owner_repo_commit)


> Entering new AgentExecutor chain...

Invoking: `web_search_query` with `{'query': 'how to create a vector store'}`
```

```
Failed to send telemetry event CollectionQueryEvent: capture() takes 1 positional argument
but 3 were given
To create a vector store in MongoDB, you need to pass your data through an embedding model to
generate vector embeddings that capture the semantic meaning of your data. These embeddings
can then be stored in a MongoDB collection as a field in a document. The dimensions of your
vector embeddings are determined by the embedding model you choose, which must be specified
in your MongoDB Vector Search index.To create a vector store in MongoDB, follow these general
steps:

1. **Generate Vector Embeddings**: Use an embedding model to transform your data into vector
embeddings. This process captures the semantic meaning of your data.

2. **Store in MongoDB**: Save these vector embeddings in a MongoDB collection as a field
within your documents.

3. **Define Index**: Create a Vector Search index in your MongoDB collection, specifying the
dimensions of your vector embeddings based on the embedding model you are using.

Make sure to consult the MongoDB documentation for specific details and examples related to
your version and setup.

> Finished chain.
Answer: To create a vector store in MongoDB, follow these general steps:

1. **Generate Vector Embeddings**: Use an embedding model to transform your data into vector
embeddings. This process captures the semantic meaning of your data.

2. **Store in MongoDB**: Save these vector embeddings in a MongoDB collection as a field
within your documents.

3. **Define Index**: Create a Vector Search index in your MongoDB collection, specifying the
dimensions of your vector embeddings based on the embedding model you are using.

Make sure to consult the MongoDB documentation for specific details and examples related to
your version and setup.

┌🍎 ankit@MacBook-Air 💻 …/AI-ML-DS/24.LangChain2/Project4 on  main [ *1 ?104 ] 🐍
(.venv) 🐍  v3.14.2  took 15s
└
```

- Here we are not using chroma or FAISS etc.

31 · LANGSERVE

LangServe is a library provided by langchain that helps developer deploy Langchain runnables and chains as a REST API or RESTful API.

This library is integrated with FASTAPI and uses pydantic for data validation.

We can install it using `pip install langserve[all]`.

uvicorn is a webserver implementation for python and this is what we used to run the server.

We use `Project5` folder here.

and create virtual environment using `python3 -m venv .venv` and then activate it using `source .venv/bin/activate` and install required packages using `python3 -m pip install -r requirements.txt`.

- we can start the server using `uvicorn server:app --reload`. Here, `--reload` is used because when any file is changed then server is automatically re-started.

So, in vscode terminal, run `uvicorn server:app --reload` and then in browser go to `http://127.0.0.1:8000/docs`.

We have to use ``pydantic==1.10.13` to use /invoke, /batch and /stream endpoints by-default.

The /invoke, /batch and /stream endpoints will work with pydantic v1 and for that python 3.10 or 3.11 is required. So, create virtual environment with these versions and then use requirements.txt file and then start the server again.

Now, in browser, use `http://127.0.0.1:8000/docs` and go to `invoke` endpoint and in request body, put:

```
{
  "input": "Tell me a joke about cats",
  "config": {},
  "kwargs": {}
}
```

Now, you get the response as:


```
{
  "output": {
    "content": "Why was the cat sitting on the computer?\n\nBecause it wanted to keep an eye
on the mouse!",
    "additional_kwargs": {},
    "response_metadata": {
      "token_usage": {
        "completion_tokens": 20,
        "prompt_tokens": 13,
        "total_tokens": 33,
        "completion_tokens_details": {
          "accepted_prediction_tokens": 0,
          "audio_tokens": 0,
          "reasoning_tokens": 0,
          "rejected_prediction_tokens": 0
        },
        "prompt_tokens_details": {
          "audio_tokens": 0,
          "cached_tokens": 0
        }
      },
      "model_name": "gpt-3.5-turbo",
      "system_fingerprint": null,
      "finish_reason": "stop",
      "logprobs": null
    },
    "type": "ai",
    "name": null,
    "id": "run-e21a7667-bfd6-487b-b0c2-1866c0f40660-0",
    "example": false,
    "tool_calls": [],
    "invalid_tool_calls": []
  },
  "metadata": {
    "run_id": "e21a7667-bfd6-487b-b0c2-1866c0f40660",
    "feedback_tokens": []
  }
}
```


- For /batch endpoint, put:

```
{
  "inputs": [
    "Tell me a joke about cats",
    "Tell me a joke about programmers"
  ],
  "config": {},
  "kwargs": {}
}
```

and you get the response as:

```
{
  "output": [
    {
      "content": "Why was the cat sitting on the computer? \n\nBecause it wanted to keep an
eye on the mouse!",
      "additional_kwargs": {},
      "response_metadata": {
        "token_usage": {
          "completion_tokens": 21,
          "prompt_tokens": 13,
          "total_tokens": 34,
          "completion_tokens_details": {
            "accepted_prediction_tokens": 0,
            "audio_tokens": 0,
            "reasoning_tokens": 0,
            "rejected_prediction_tokens": 0
          },
          "prompt_tokens_details": {
            "audio_tokens": 0,
            "cached_tokens": 0
          }
        },
        "model_name": "gpt-3.5-turbo",
        "system_fingerprint": null,
        "finish_reason": "stop",
        "logprobs": null
      },
      "type": "ai",
      "name": null,
      "id": "run-10f6a822-43e1-43af-87c9-43301b079d3f-0",
      "example": false,
      "tool_calls": [],
      "invalid_tool_calls": []
    },
    {
      "content": "Why do programmers prefer dark mode? Because the light attracts bugs!",
      "additional_kwargs": {},
      "response_metadata": {
        "token_usage": {
          "completion_tokens": 13,
          "prompt_tokens": 13,
          "total_tokens": 26,
          "completion_tokens_details": {
            "accepted_prediction_tokens": 0,
            "audio_tokens": 0,
            "reasoning_tokens": 0,
            "rejected_prediction_tokens": 0
          },
          "prompt_tokens_details": {
            "audio_tokens": 0,
            "cached_tokens": 0
          }
        },
        "model_name": "gpt-3.5-turbo",
        "system_fingerprint": null,
        "finish_reason": "stop",
        "logprobs": null
```

```
      },
      "type": "ai",
      "name": null,
      "id": "run-fa9a316f-28e6-477d-b1f5-adacb87ea5b8-0",
      "example": false,
      "tool_calls": [],
      "invalid_tool_calls": []
    }
  ],
  "metadata": {
    "responses": [
      {
        "run_id": "10f6a822-43e1-43af-87c9-43301b079d3f",
        "feedback_tokens": []
      },
      {
        "run_id": "fa9a316f-28e6-477d-b1f5-adacb87ea5b8",
        "feedback_tokens": []
      }
    ],
    "run_ids": [
      "10f6a822-43e1-43af-87c9-43301b079d3f",
      "fa9a316f-28e6-477d-b1f5-adacb87ea5b8"
    ]
  }
}
```


32. Now, we create more routes with runnables.

Now, restart the server again and you will see the endpoint for `joke` also.

Now, open another terminal and put:

```
curl --location --request POST 'http://localhost:8000/joke/invoke' \
    --header 'Content-Type: application/json' \
    --data-raw '{
        "input": {
            "topic": "cats"
        }
    }'
```

You will get the output as:


```
{"output":"Why was the cat sitting on the computer?\n\nBecause it wanted to keep an eye on the mouse!","metadata":{"run_id":"8d571431-6bdc-4938-935f-51f2857d3f6d","feedback_tokens":[]}}
```


In the joke/invoke endpoint in browser also, you can use:


```
{
  "input": {
    "topic": "honey"
  },
  "config": {},
  "kwargs": {}
}
```


33. You can add vector search root also. So, run again the server and go to query/invoke endpoint and ask query as:

```
{
  "input": "What are the security best practices for MongoDB Atlas?",
  "config": {},
  "kwargs": {}
}
```

You will get the output as:

```
{
  "output": "The security best practices for MongoDB Atlas include:\n- enable strong
authentication\n- restrict IP access\n- use least privilege roles\n- enable encryption in
transit and at rest",
  "metadata": {
    "run_id": "8120ad59-eab2-4d64-ad01-890b72c5bed7",
    "feedback_tokens": []
  }
}
```

34. `Render(https://render.com/)` is cloud application hosting platform for developers.

It provides an easy way to build, scale and deploy apps in production without hassle.

Create an account on render using github and create web services by pulling github repo.

Use `Project6/RAG-REST-API` folder for deployment here.

- First go to `Project6/RAG-REST-API` location in vscode terminal.
- then type `git init`
- Now, go to github and create a remote repo with name as `rag-rest-webservice` with not selecting readme as it is already created locally.
- Now, type `git add .`
- Now, type `git commit -m "first commit"`
- Now, type `git remote add origin https://github.com/ankitgupta1729/rag-rest-webservice.git`
- Now, at the end, type `git push -u origin main`.

Now, we pull this github repo in render. So, we create a new web service in render. So, go to render and select `New web service` under `Web Service`.

Now, select git provider as github and add your repo there.

Now, put the start command as `uvicorn main:app --host 0.0.0.0` and then go to Environment variables and put all the things from .env as key-value pair like `OPENAI_API_KEY` as variable name and put its value..

At the end, click on `Deploy Web Service` and at the end after build successful, check the URL displyed on the page by apending `docs` at the end i.e. `https://rag-rest-webservice.onrender.com/docs` with `joke` endpoint and topic as `python developers`.